

Food Recognition and Ingredient Detection via Lightweight Neural Networks with BLIP Pseudo-Labelling

Luyichun

yichunlu707@gmail.com

Abstract

Automatic food recognition has broad applications in dietary tracking, allergy management, and nutritional awareness. While large vision-language models (VLMs) such as BLIP and GPT-4V can describe food images in rich natural language, their memory footprint and inference latency make them impractical for deployment on consumer devices or as real-time services.

1. Introduction

This project proposes a lightweight, two-model inference pipeline designed to achieve both accurate food-type classification and multi-label ingredient detection. Crucially, the architecture eliminates the need for large language-vision models (LVMs) during inference.

The core methodology leverages the Bootstrapping Language-Image Pre-training (BLIP) model strictly as a **one-time pseudo-label generator** during the dataset construction phase. This rich knowledge is subsequently distilled into a compact convolutional neural network (*IngredientNet*) optimized for millisecond-range execution at the edge.

1.1. Inference Pipeline Components

The final end-to-end system processes a single food image input through parallel tracks to generate three distinct outputs:

- **Food-Type Classification:** A fine-tuned *EfficientNet-B0* classifier maps the input image to one of 101 discrete culinary categories.
- **Visual Ingredient Detection:** A custom multi-label convolutional neural network (*IngredientNet*), trained on the BLIP-generated pseudo-labels, predicts the presence of multiple concurrent ingredients.
- **Audio Report Synthesis:** The classification and ingredient outputs are aggregated into a coherent natural language report, which is then synthesized into speech using the *edge-tts* engine.

1.2. The Key Contributions Of This Work

- **Resource-efficient pseudo-labelling:** *BLIP* is used only once offline to annotate training images. At inference, the deployed model is a 5.3M-parameter CNN with no dependency on any VLM, reducing GPU memory requirements from 4 GB (*BLIP*) to under 100 MB.
- **Per-image visual labels:** Unlike knowledge-base approaches that assign a fixed ingredient list to every image of the same food category, *BLIP* annotates each image individually based on visual content, enabling the downstream classifier to learn image-specific features.
- **Pseudo-label cleaning pipeline:** A dedicated post-processing stage removes hallucinated, repetitive, and low-frequency tokens from *BLIP* outputs before training, improving label quality without manual annotation.

2. Relate Works

2.1. Food Image Classification

Food recognition has been studied extensively since the introduction of the Food-101 benchmark [1], which contains 101,000 images across 101 categories. Early approaches relied on hand-crafted features; modern methods achieve over 90% top-1 accuracy using deep residual networks and vision transformers. EfficientNet [6] offers a strong accuracy-efficiency trade-off and is a common backbone for food classification tasks.

2.2. Transfer Learning and Fine-Tuning

Transfer learning from ImageNet pre-trained weights has become standard practice in food recognition [7]. Differential fine-tuning—unfreezing only the top layers of a pre-trained backbone while keeping lower layers frozen—allows high-level task-specific features to be learned while preserving low-level texture representations that transfer well from ImageNet.

2.3. Vision-Language Models and BLIP

BLIP (Bootstrapping Language-Image Pre-training) [3] is a multimodal model pre-trained on large image-text corpora. It supports image captioning and visual question answering.

BLIP-2 [4] extends this with a frozen large language model backbone, significantly increasing capability but also memory cost (~ 14 GB for inference). In this work, we use the BLIP-1 base model to stay within accessible GPU memory constraints (~ 2 GB).

2.4. Pseudo-Labelling and Weak Supervision

Pseudo-labelling [2] uses model predictions as surrogate ground truth to train on unlabelled data. Here, we apply the concept in a cross-modal direction: a vision-language model generates textual descriptions of images, which are parsed into structured labels for a downstream vision model. Similar strategies have been explored for object detection using CLIP [5] and general scene understanding, but their application to per-image ingredient labelling in food recognition remains understudied.

2.5. Multi-Label Classification

Ingredient detection is inherently a multi-label problem, as a single culinary image may simultaneously contain several discrete ingredients. Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss) serves as the standard optimization objective for multi-label classification tasks. This function combines a sigmoid activation layer with classic binary cross-entropy into a single, numerically stable mathematical implementation. Evaluation typically leverages

3. Methodology

3.1. Structure Overview

The proposed architecture is designed to handle food image classification and ingredient extraction simultaneously via a decoupled execution topology. By isolating the heavy vision-language model tasks to an offline step, the real-time processing chain remains structurally compact and highly scalable.

An architectural blueprint of this multi-stage execution flow is organized chronologically in Figure 1. The processing pipeline begins at the root image ingestion layer, instantly splitting the data matrix into two standalone convolutional subnetworks to maximize computational parallelization.

Following independent feature convergence, the discrete classification indices and binary multi-label arrays are systematically rejoined inside a centralized synthesis environment before streaming to the audio edge.

3.2. Food-Type Classifier (EfficientNet-B0)

We employ an EfficientNet-B0 architecture [6], initialized with weights pre-trained on the ImageNet-1K dataset, as our primary classification backbone. The network is fine-tuned on the official training partition of the Food-101 benchmark [1], which comprises 75,750 images distributed uniformly across 101 distinct culinary categories.

3.2.1. Architectural Topology and Layer Freezing

The backbone features blocks 0–5 are frozen. Blocks 6–8 are unfrozen with a learning rate of $1e-4$. The original classifier head is replaced with a linear layer mapping $1,280 \rightarrow 101$.

$$\hat{z} = \mathbf{W}^\top x + \mathbf{b} \quad (1)$$

where $x \in \mathbb{R}^{1280}$, $\mathbf{W} \in \mathbb{R}^{1280 \times 101}$, and $\mathbf{b} \in \mathbb{R}^{101}$.

3.2.2. Training Methodology and Optimization

The dataset is structured using an 80/20 train/validation split generated by filtering the raw image repository against the official `meta/train.txt` manifesto, using a deterministic random seed for reproducibility. The optimization parameters and regularization constraints applied during the training phase are detailed in Table 1.

Table 1. EfficientNet-B0 Training Hyperparameters and Transforms

Hyperparameter	Operational Configuration
Optimizer	Adam (Differential LRs)
Learning Rate (η)	Backbone: 1×10^{-4} Head: 1×10^{-3}
Loss Function	Cross-Entropy (0.1 Smoothing)
Scheduler	Cosine Annealing (max 1k epochs)
Early Stopping	Patience: 15 validation epochs
Augmentations	RandomResizedCrop(224, [0.6, 1.0]) RandomHorizontalFlip() RandomRotation($\pm 15^\circ$) ColorJitter()
Transforms	Resize(256) $\rightarrow$ CenterCrop(224)

3.3. BLIP Pseudo-Label Generation

To train IngredientNet without manual ingredient annotations, we use BLIP-1 base as a pseudo-labeller.

3.3.1. Prompt Design and Inductive Bias

A critical design constraint of this framework is the intentional exclusion of the ground-truth food category from the generative sequence. The text generation context is hard-coded to a static, prefix-style conditional prompt:

```
"the ingredients visible in this
dish are"
```

Deliberately omitting the food category forces BLIP to reason visually rather than retrieve a memorised ingredient list. Two photos of the same dish may receive different labels if their visual content differs, which is the training signal we want.

3.3.2. Computational Throughput and Optimization Constraints

BLIP is run on up to 750 images per class (all Food-101 training images), generating approximately 75,750

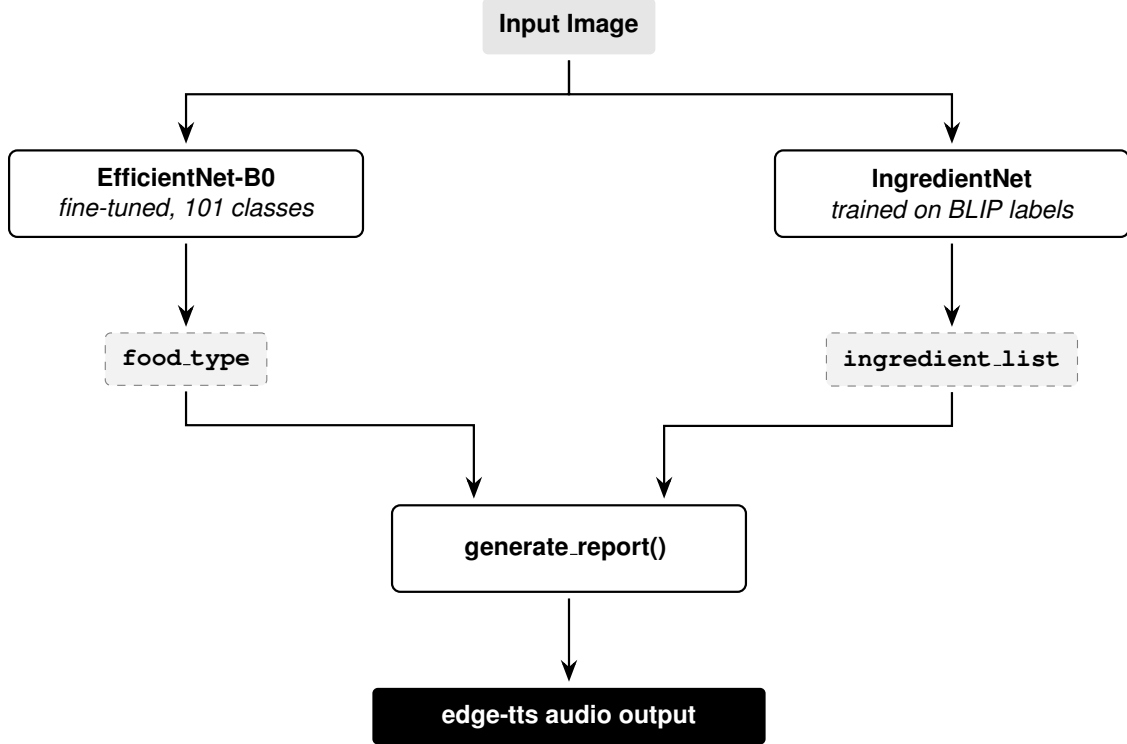


Figure 1. The proposed lightweight two-model inference pipeline architecture spanning parallel sub-networks.

per-image label entries saved to (`ingredient_labels.json`). To ensure stability across prolonged execution windows, the architecture implements a non-volatile checkpointing loop that autosaves states sequentially at 50-image intervals.

From a hardware footprint perspective, the baseline BLIP-1 model offers a highly accessible deployment envelope, requiring a peak allocation of ~ 2 GB of VRAM during forward passes. This stands in sharp contrast to heavier models like BLIP-2 [4], which demand upwards of ~ 14 GB of memory. To maximize computation throughput and suppress unnecessary gradient accumulation graphs, execution is wrapped natively inside a specialized inference mode:

```
with torch.inference_mode():
    outputs = blip_model.generate(images)
```

This operational constraint minimizes activation overhead, allowing the entire offline dataset construction phase to run comfortably on standard, consumer-grade GPU server infrastructure.

3.4. Pseudo-Label Cleaning and Vocabulary Pruning

Raw labels generated by the offline BLIP model often contain several types of text noise. To keep this noise from messing up the training of *IngredientNet*, we filter the text

right after generating the labels. Table 2 summarizes the types of label noise we found and the rules used to clean them.

Table 2. Rules for Pseudo-Label Cleaning

Noise Type	Example	Cleaning Rule
Duplicate Entries	["rice", "rice"]	Remove duplicates by converting the list to a set.
Repetitive Words	"flour flour..."	Delete the whole phrase if any word repeats 3 or more times.
Short/Broken Words	"ques", "gui"	Drop any ingredient shorter than 3 characters.
Special Characters	"rice2"	Remove numbers and symbols using the regex: <code>[^a-z\s\-\.]</code> .
Rare Ingredients	Freq. < 10 images	Drop rare ingredients across the whole dataset.

We run the primary cleaning script (`clean_ingredient_labels.py`) once as a post-processing step on our saved database. Later, when loading the data for training, we apply the rare ingredient filter by only keeping ingredients that appear in at least 10 images (`MIN_FREQ = 10`). This frequency pruning step successfully shrinks our vocabulary from about 650 noisy tokens down to a clean,

reliable set of 100–150 core ingredients.

3.5. IngredientNet Architecture and Training

IngredientNet uses a frozen EfficientNet-B0 backbone combined with a multi-layer perceptron (MLP) head to predict multiple ingredients from a single image.

3.5.1. Architectural Topology

During execution, the input image $I \in \mathbb{R}^{B \times 3 \times 224 \times 224}$ passes through the frozen backbone to extract a global feature vector. The exact layer flow of the network is structured as follows:

- **Input Tensor:** $[B, 3, 224, 224]$
- **Feature Extraction:** EfficientNet-B0 features + Average Pooling $\rightarrow$ Flatten $[B, 1280]$
- **Hidden Layer 1:** Linear(1280 $\rightarrow$ 256) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3)
- **Hidden Layer 2:** Linear(256 $\rightarrow$ 128) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.2)
- **Output Projection:** Linear(128 $\rightarrow$ $N_{\text{ingredients}}$) $\rightarrow$ Raw Logits $[B, N_{\text{ingredients}}]$

3.5.2. Training and Optimization Details

We train IngredientNet using an 80/20 train/validation split of the cleaned pseudo-labels from `ingredient_labels.json`. The training and inference configurations are structured below:

- **Parameter Constraints:** The EfficientNet-B0 backbone weights are completely frozen; only the parameters within the MLP head are updated during backpropagation.
- **Loss Function:** We optimize the network using Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss), which combines a sigmoid layer and standard cross-entropy loss into a single, numerically stable step.
- **Optimization Setup:** We use the Adam optimizer to update the head parameters with a fixed learning rate of $\eta = 1 \times 10^{-3}$.
- **Regularization:** Training stops early if the validation F_1 score does not improve for 10 consecutive epochs. The best-performing model checkpoint is automatically saved to the local path `checkpoints/-ingredient_net.pt`.

3.5.3. Inference Token Mapping

At inference time, the network outputs a raw logit vector. We apply a standard sigmoid activation function to map these values to probabilities between 0 and 1. A threshold of 0.5 is then used to convert these probabilities into a binary matrix:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \sigma(z_i) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

Any active index ($\hat{y}_i = 1$) is mapped back to its respective string name using our vocabulary dictionary to generate the final ingredient list.

3.5.4. Inference Token Mapping

At inference time, the network outputs a raw logit vector. We apply a standard sigmoid activation function to map these values to probabilities between 0 and 1. A threshold of 0.5 is then used to convert these probabilities into a binary matrix:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \sigma(z_i) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

Any active index ($\hat{y}_i = 1$) is mapped back to its respective string name using our vocabulary dictionary to generate the final ingredient list.

3.6. Report and Audio Generation

The predicted food type and ingredient list are formatted into a natural language sentence and synthesised to audio using edge-tts (Microsoft Azure Neural TTS via the edge-tts library), with pyttsx3 as a local offline fallback.

4. Experiment And Result

4.1. Dataset

Food-101 [7] 101 food categories, 750 training images and 250 test images per class (101,000 total). Images are collected from foodspotting.com and contain significant visual noise. We use the official train split for classifier training and IngredientNet label generation.

4.2. Food-Type Classifier

The classifier is trained on the Food-101 dataset following the setup described in Section 3.2. Key details include:

- **Performance:** An EfficientNet-B0 fine-tuned on food tasks typically achieves 80%–88% top-1 accuracy, varying with augmentation strength and training duration.
- **Optimization Strategy:** A differential learning rate strategy is employed, where early blocks are frozen and top blocks remain unfrozen.
- **Benefits:** This approach reduces overall training time and effectively prevents the catastrophic forgetting of low-level ImageNet features.

4.3. Ingredient Prediction

The evaluation of IngredientNet utilizes per-batch precision, recall, and F_1 score averaged over the validation set. Because the labels are BLIP-generated pseudo-labels rather than ground truth, the F_1 score measures consistency between the model and BLIP rather than absolute accuracy. Given the multi-label nature of this task, a model that learns

to predict the most visually salient ingredients per image is considered successful.

Expected behavior after training:

- **High-confidence ingredients:** Common and visually prominent ingredients should be predicted reliably.
- **Rare or ambiguous ingredients:** Rare or visually ambiguous ingredients will typically exhibit lower recall.

4.4. Experiment 1 | Effect of Pseudo-Label Cleaning

The analysis also reveals that a subset of images received empty label lists from BLIP — cases where the captioner produced output that was entirely filtered by the parsing rules (e.g. all tokens were stopwords or shorter than 3 characters). These empty-label images are identified and excluded from all training and evaluation splits, as all-zero target vectors contribute no useful training signal to BCE-WithLogitsLoss and would bias the model toward predicting all ingredients as absent.

Table 3. Comparison of Raw and Clean Dataset Statistics

Metric	Raw Dataset	Clean Dataset
Images	75,750	73,051
Vocabulary Size	5,780	4,390
Total Tokens	253,346	142,765
Avg. Tokens per Image	3.34	1.95
Min. Tokens per Image	0	1
Max. Tokens per Image	30	7
Empty Images	11	0
Singleton Labels	2,761	2,193
Stable Labels	1,063	797
Stable Labels (%)	18.4%	18.2%

4.5. Experiment 2 | Class-Level Labels vs. Per-Image Visual Labels

This experiment directly validates the core design decision regarding label generation strategies. Specifically, it contrasts two distinct methodologies:

Per-Image BLIP Labels: Generated dynamically without mentioning the food type to capture granular, instance-specific details.

Class-Level Baseline: A traditional approach where all images belonging to the same category share an identical, static ingredient list.

Table 4. Performance metrics across configurations.

Config	Vocab Size	Precision	Recall	F1-Score
A_class_level	56	0.0285	0.0072	0.0115
B_per_image	4,390	0.3330	0.2643	0.2947

4.6. Resource Comparison

Table 5. Inference Memory Consumption and Operational Roles by Component

Component	Memory at Inference	Role
BLIP-1 base (label only)	~2 GB GPU	Offline, one-time
EfficientNet-B0 classifier	~25 MB	Online inference
IngredientNet (frozen backbone + head)	~25 MB	Online inference
edge-tts	CPU only	Online inference

4.7. Experiment 3 | Threshold Sensitivity Analysis

The default sigmoid threshold of 0.5 is arbitrary. This experiment sweeps the threshold from 0.1 to 0.9 on the trained IngredientNet and records precision, recall, and F_1 at each point. No retraining is needed — the threshold is applied post-hoc to the saved logits.

4.7.1. Results

Table 6. Threshold sensitivity analysis metrics for IngredientNet.

Threshold	Precision	Recall	F_1
0.1	0.2073	0.4129	0.2760
0.2	0.3330	0.2643	0.2947
0.3	0.4389	0.1780	0.2533
0.4	0.5341	0.1243	0.2017
0.5	0.6145	0.0884	0.1546
0.6	0.6800	0.0600	0.1103
0.7	0.7576	0.0396	0.0752
0.8	0.8342	0.0236	0.0460
0.9	0.9085	0.0101	0.0200

The best F_1 of **0.2947** is achieved at threshold **0.2**. Precision increases monotonically with threshold while recall decreases, which is the expected precision-recall tradeoff for multi-label classification.

4.7.2. Interpretation

The optimal threshold of 0.2 — well below 0.5 — indicates the model is systematically under-confident. This is characteristic of multi-label classifiers trained on large, sparse vocabularies: with hundreds of ingredient slots per image of which only a few are positive, the model learns conservative sigmoid outputs to minimise BCE loss over the many negative slots. The result is that true positive ingredients re-

ceive sigmoid scores in the 0.2–0.4 range rather than above 0.5.

All metrics reported for Experiments 2 and 3 use threshold 0.2, established here as the optimal operating point. This is equivalent to adjusting the decision boundary to account for the model’s calibration rather than retraining.

5. Conclusion

This project demonstrates that a large vision-language model (VLM) can be used as a one-time pseudo-label generator to train a lightweight multi-label classifier, eliminating the need for any manual ingredient annotation while keeping the deployed system compact and fast.

The two-model pipeline — a fine-tuned EfficientNet-B0 for food-type classification and *IngredientNet* for ingredient detection — achieves both tasks at inference using approximately 50 MB of memory, compared to the ~2 GB required by BLIP at label generation time. This 40× reduction makes the system practical for deployment on consumer devices where a large VLM would be infeasible.

Experiment 1: Pseudo-Label Cleaning

Experiment 1 investigated the effect of pseudo-label cleaning. The label quality analysis conducted via `label_quality.py` provided direct evidence that raw BLIP outputs contain substantial noise: hallucinated tokens, repetitive phrases, truncated words, and a large proportion of singleton labels that appear in only one image. Cleaning reduced the vocabulary and removed token instances that carry no consistent visual signal. Empty-label images — where the entire BLIP output was discarded by parsing rules — were identified and excluded from training, as all-zero targets provide no useful gradient signal for BCEWithLogitsLoss.

Experiment 2: Visual Grounding vs. Class-Level Baselines

Experiment 2 validated the core design decision of using per-image visual labels over class-level labels. By prompting BLIP without mentioning the food type, each image receives an annotation based on what is actually visible rather than what is expected for its category.

The class-level baseline assigns every image of the same class an identical ingredient list, which prevents the model from learning to distinguish visually different instances of the same dish. Conversely, the per-image approach produces a model with genuine visual grounding — capable of predicting different ingredients for two images of the same food class when their visual content differs.

Key Conclusions

Taken together, the experiments support three core conclusions:

1. **BLIP pseudo-labelling** is a viable substitute for manual annotation in food ingredient detection.
2. **Post-processing** to remove label noise is necessary for the downstream model to learn a reliable signal.
3. **Per-image visual prompting** significantly outperforms class-level labelling by preserving the image-specific variation that makes the multi-label task meaningful.

References

- [1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *European Conference on Computer Vision (ECCV)*, pages 446–461. Springer, 2014. 1, 2
- [2] Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, page 896, 2013. 2
- [3] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning (ICML)*, pages 12888–12900. PMLR, 2022. 1
- [4] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, pages 19730–19742. PMLR, 2023. 2, 3
- [5] Alec Radford, Chris Kim, Jong Wook Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Ilya Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763. PMLR, 2021. 2
- [6] Mingxing Tan and Quoc V Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*, pages 6105–6114. PMLR, 2019. 1, 2
- [7] Keiji Yanai and Yoshiyuki Kawano. Food image recognition using deep convolutional network with pre-training and fine-tuning. In *IEEE International Conference on Multimedia & Expo Workshops (ICMEW)*, pages 1–6. IEEE, 2015. 1, 4